

Mini-project

Computer Vision

Nguyễn Thế Vũ

October 15, 2024

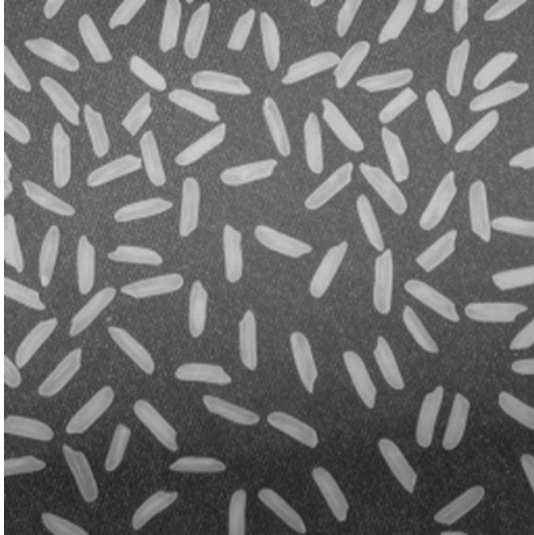
Mục lục

1. Đếm số đối tượng	3
1.1. Mục tiêu bài toán	3
1.2. Quy trình xử lý	3
1.2.1. Tiền xử lý	3
1.2.2. Xử lý chính	5
1.2.3. Hậu xử lý	5
1.3. Kết quả	6
2. Cắt khung ảnh	8
2.1. Mục tiêu bài toán	8
2.2. Quy trình xử lý	8
2.2.1. Tiền xử lý	8
2.2.2. Xử lý chính	9
2.2.3. Hậu xử lý	10
2.3. Kết quả	10
3. Content-based Retrieval Image (CBIR)	11
3.1. Bài toán	11
3.2. Nghiên cứu liên quan	11
3.3. Phương pháp	11
3.3.1. Ý tưởng	11
3.3.2. Các phương pháp vector hóa	12
3.3.3. Vector hóa với Histogram màu	12
3.3.4. Vector hóa với Histogram cạnh	13
3.3.5. Vector hóa bằng phương pháp Fourier Descriptor	13
3.3.6. Vector hóa dựa trên bộ mô tả SIFT	14
3.3.7. Vector hóa hình ảnh bằng Resnet	16
3.4. Thực nghiệm	17
3.4.1. Bộ dữ liệu	17
3.4.2. Phương pháp thực nghiệm	17
3.4.3. Tham số thực nghiệm	18
3.4.4. Kết quả thực nghiệm	19
3.5. Kết luận	23
Tài liệu tham khảo	24
Index of Figures	25

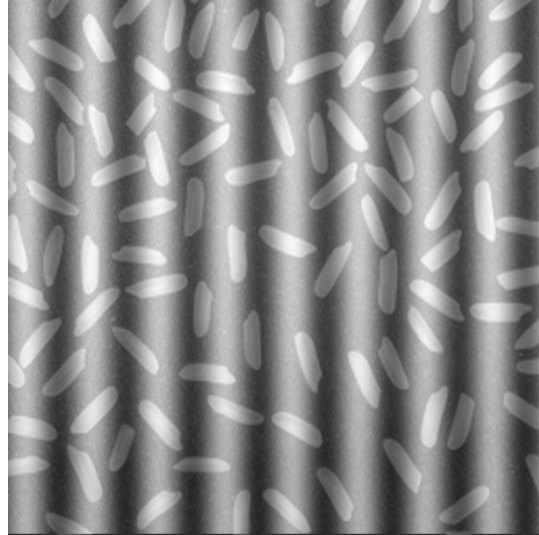
1. Đếm số đối tượng

1.1. Mục tiêu bài toán

Đếm số đối tượng xuất hiện trong ảnh. Các ảnh đầu vào là hình ảnh các hạt gạo trong các điều kiện ảnh khác nhau:



Hình 1: Ảnh gốc



Hình 2: Ảnh có nhiễu sọc trắng đen



Hình 3: Ảnh có nhiễu hạt tiêu



Hình 4: Ảnh có cường độ sáng không đều

1.2. Quy trình xử lý

Quy trình xử lý sẽ được chia thành 3 phần:

- Tiền xử lý: Tăng cường chất lượng ảnh, lọc nhiễu để tăng cường hiệu quả cho các bước sau.
- Xử lý chính: Chuyển đổi ảnh sang không gian khác và xử lý để tìm ra được kết quả cuối cùng.
- Hậu xử lý: Từ ảnh đã được xử lý, đọc và trả ra kết quả.

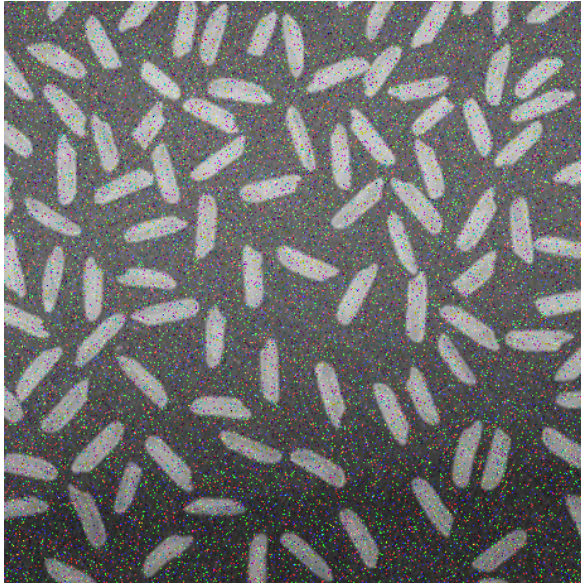
Chi tiết các bước xử lý sẽ được trình bày sau đây.

1.2.1. Tiền xử lý

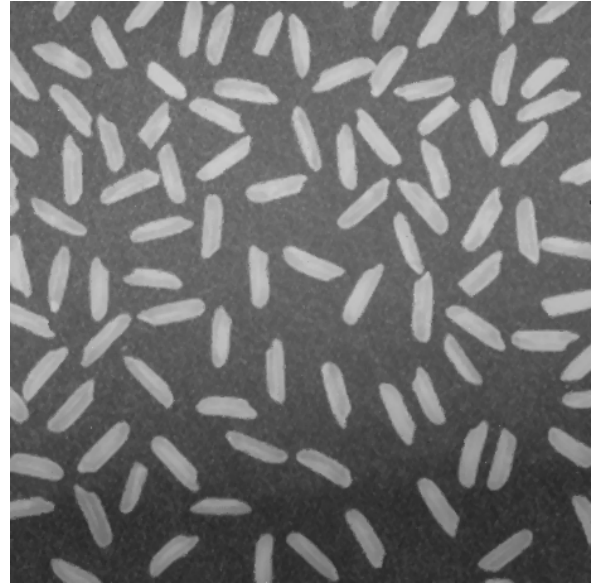
Trong phần tiền xử lý, ảnh sẽ được xử lý qua các bước sau:

- Chuyển đổi ảnh từ không gian màu RGB sang Grayscale để dễ dàng xử lý

- Sử dụng bộ lọc Median với `kernel_size=3` để lọc nhiễu đặc biệt là nhiễu hạt tiêu.

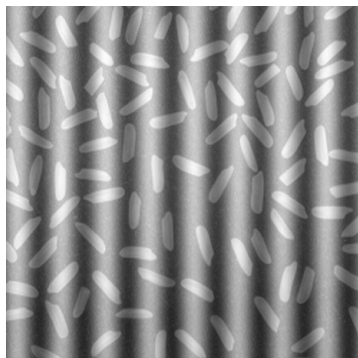


Hình 5: Ảnh trước khi lọc nhiễu

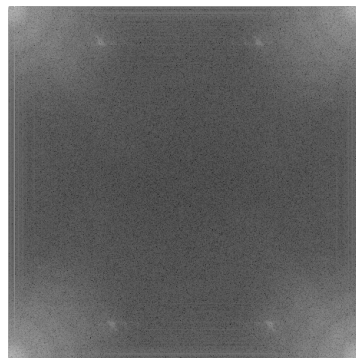


Hình 6: Ảnh sau khi lọc nhiễu

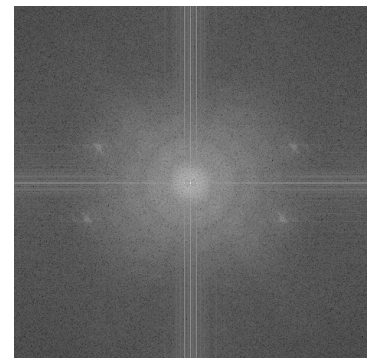
- Sử dụng Fourier Transform để chuyển đổi ảnh sang miền tần số để thực hiện lọc nhiễu sọc trắng, đen.
 - Khi thực hiện phép biến đổi Fourier Transform. Ta thu được ảnh trên miền tần số, các trục trên ảnh này biểu thị tần số của sự biến đổi các pixel ảnh và giá trị biểu thị biên độ biến đổi. Thực hiện phép chuyển gốc tọa độ vào giữa, ta được ảnh như sau:



Hình 7: Ảnh có nhiễu sọc đen trắng

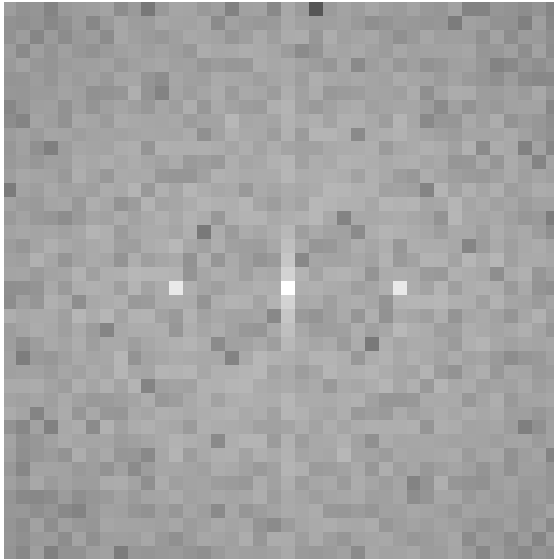


Hình 8: Miền tần số của ảnh

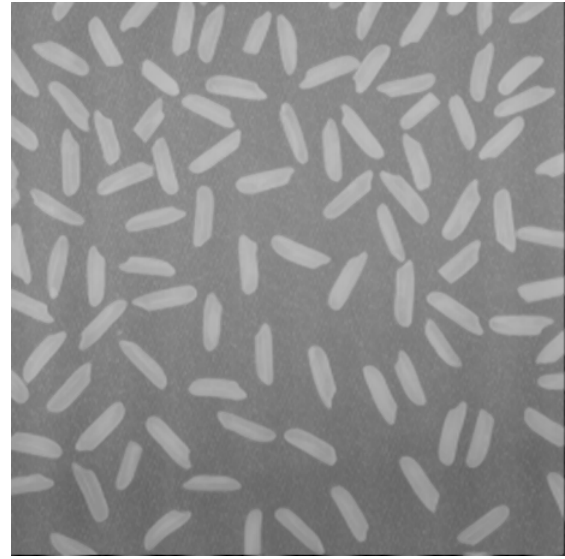


Hình 9: Miền tần số của ảnh khi chuyển gốc tọa độ về trung tâm

- Khi phóng to phần trung tâm của ảnh Hình 9. Ta sẽ thấy có 3 vùng tập trung điểm chấm trắng, 1 điểm ở trung tâm và 2 điểm trên trục x nằm đối xứng nhau qua trung tâm. Ảnh miền tần số sáng ở giữa và tối về bên rìa cho thấy tần số càng thấp thì biên độ càng cao. Tuy nhiên khi xem xét có 2 điểm trên trục x tự dung cao 1 cách bất thường, điều này cho thấy là tín hiệu biến đổi theo tần số đó là rất rõ rệt. Vì vậy khi ta loại bỏ sự biến đổi tín hiệu trên tần số này, ta sẽ lọc được nhiễu sọc vốn dĩ là sự biến đổi từ trắng sang đen theo 1 chu kì cố định.



Hình 10: Zoom vào trung tâm miền tần số



Hình 11: Ảnh sau khi lọc nhiễu sọc

- Tiếp theo, để cải thiện chất lượng của ảnh trước khi xử lý, ảnh sẽ được tăng sáng và tăng độ tương phản bằng phương pháp CLAHE (Contrast Limit Adaptive Histogram Equalizer). Phương pháp CLAHE được lựa chọn để tăng độ tương phản vừa để đảm bảo các chi tiết cục bộ vừa để hạn chế nhiễu khi tăng.

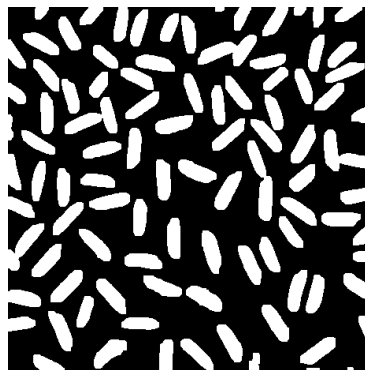
1.2.2. Xử lý chính

Ảnh sẽ được xử lý theo các bước sau:

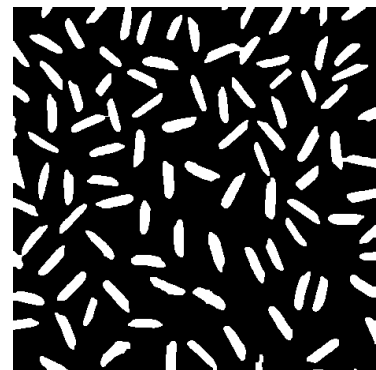
- Đầu tiên để việc đếm các vật thể có thể được diễn ra, ảnh sẽ được chuyển đổi từ ảnh đen trắng sang ảnh nhị phân. Ở đây, vì có ảnh có độ sáng không đều, phương pháp lấy ngưỡng thích nghi (adaptive threshold) được lựa chọn để tối ưu độ chính xác.
- Ảnh nhị phân sẽ được áp dụng phương pháp biến đổi hình thái để nâng cao chất lượng. Cụ thể, phép mở (Open) được sử dụng để lọc đi những vật thể quá nhỏ, thường là những điểm nhiễu và phép co (Erode) được sử dụng để co và phân tách các vật thể bị dính với nhau.



Hình 12: Ảnh nhị phân



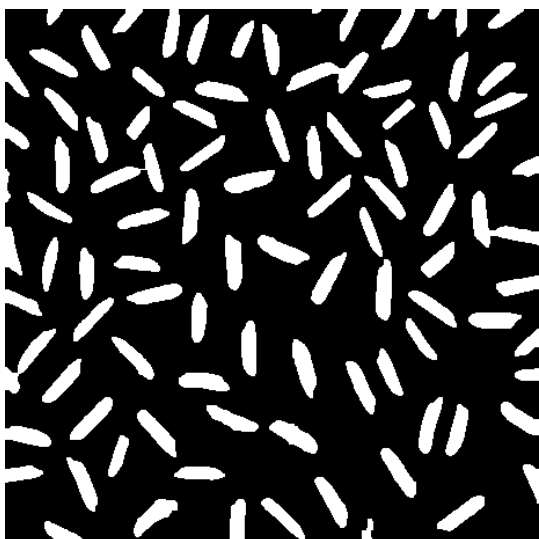
Hình 13: Ảnh nhị phân sau khi thực hiện phép mở



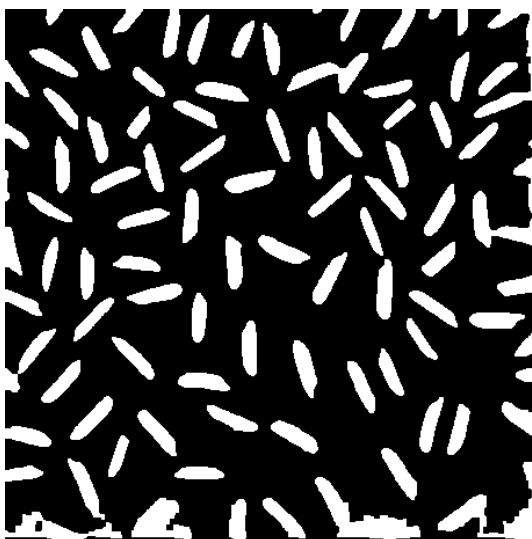
Hình 14: Ảnh nhị phân sau khi thực hiện phép co

1.2.3. Hậu xử lý

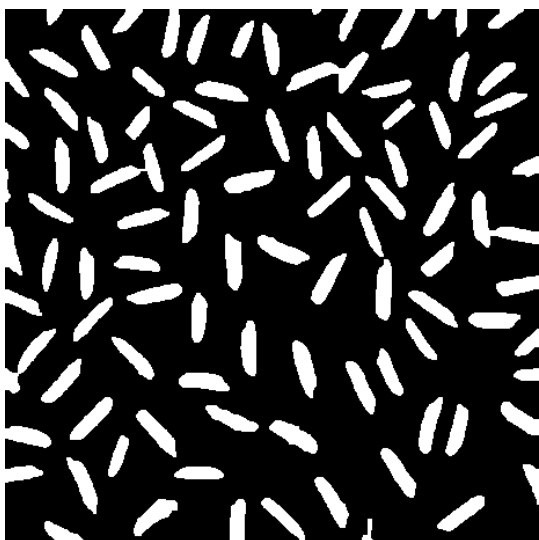
Phương pháp đánh nhãn thành phần liên thông được sử dụng để đánh nhãn từng đối tượng từ đó đếm ra được số đối tượng. Hình sau là kết quả của các hình ảnh sau khi xử lý:



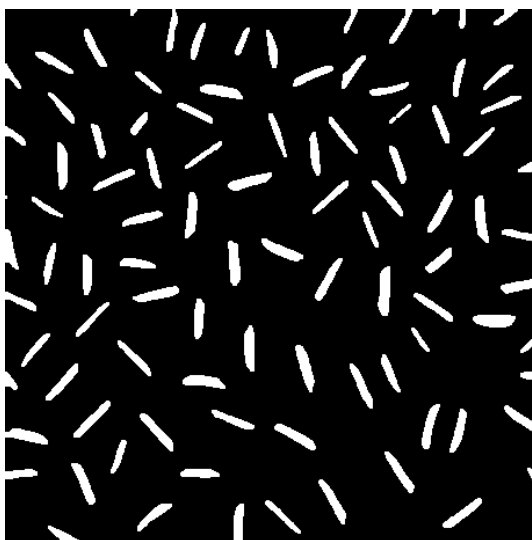
Hình 15: Ảnh đã qua xử lý của ảnh thường Hình 1



Hình 16: Ảnh đã qua xử lý của ảnh sọc trắng đen Hình 2



Hình 17: Ảnh đã qua xử lý của ảnh nhiễu hạt tiêu Hình 3



Hình 18: Ảnh đã qua xử lý của ảnh sáng không đều Hình 4

1.3. Kết quả

Kết quả phát hiện số lượng đối tượng của từng ảnh như sau:

- Hình 15 phát hiện được 100 đối tượng.
- Hình 16 phát hiện được 100 đối tượng.
- Hình 17 phát hiện được 96 đối tượng.
- Hình 18 phát hiện được 99 đối tượng.

Phân tích kết quả thu được:

- Hình 15, Hình 16, Hình 17 vẫn còn có vật thể bị dính lại với nhau (góc trên). Điều này do co chưa đủ nhưng khi áp dụng phép co quá nhiều thì những vật thể ở rìa cũng bị biến mất.
- Hình 16 vẫn còn bị nhiễu ở phần rìa. Điều này xảy ra do ảnh vẫn còn 1 đoạn bị tối sáng ở phần rìa phải,

- Hình 18 một vài vật thể ở rìa bị mất và một vật thể bị tách ra làm hai vật thể nhỏ do các vật thể trong ảnh tương đối nhỏ và phân tách.

→ Nhìn chung để sử dụng một pipeline chung để xử lý cho toàn bộ ảnh khá là khó và còn nhiều điểm trên từng ảnh riêng biệt chưa xử lý được. Trong tương lai, có thể phát triển bộ nhận diện và thay đổi tham số tự động trên từng kiểu ảnh để tối ưu kết quả.

2. Cắt khung ảnh

2.1. Mục tiêu bài toán

Tách các ô có ghi các bước đi (viết tay) trong ảnh.

1	d4	Nf6	31	g4	h4
2	c4	e6	32		
3	Nc3	d5	33		
4	Nf3	Bb4	34	Black	
5	Bd2	O-O	35	Resigns	
6	e3	Nc6	36		
7	Bd3	Bd7	37		
8	O-O	Qd7	38		
9	a3	dxc4	39		
10	Oxc4	Qd6	40		
11	Nb5	Ng4	41		
12	h3	Nf6	42		
13	Rc1	a6	43		
14	Kxd6	cxd6	44		
15	Rd4	b5	45		
16	Bd3	d5	46		
17	Nc5	Pf6	47		
18	Nxc6	Oxc6	48		
19	f3	Rd7	49		
20	Qd4	Rc7	50		
21	e5	Nd7	51		
22	b4	Qd6	52		
23	Qd6	Rd7	53		
24	Qd2	Nb6	54		
25	b3	Rd7	55		
26	Bc5	Qd8	56		
27	Rc3	a5	57		
28	Rc1	Rc8	58		
29	Bxb6	Oxb6	59		
30	Oxd4	Oxd4	60		

Hình 19: Ảnh ván cờ 1

1	d4	Nf6	31	Kg1	Kc4
2	c4	d5	32	Kc3	Kd6
3	Nc3	d5	33	Nc2	f5
4	Nf3	Bg7	34	gxf5	gxf5
5	e3	O-O	35	Ng3	Ke6
6	h3	C5	36	Ke2	Ke6
7	dxc4	Qxc4	37	f3	Kg5
8	Bd3	Qxc4	38	Kf3	h6
9	g3	Nc4	39	Kf4	Kg5
10	Rc1	Nxc3	40	Ke2	Kd6
11	Bxc3	Bxc3	41	Kd2	Kd6
12	Rxc3	Qxc3	42	Ke2	
13	Qd2	Rd8	43		
14	Rc1	Qxd4	44		
15	Nxc2	d4	45		
16	Qxd4	Nxc2	46		
17	Nf3	Rd3	47		
18	Kh1	Nc6	48		
19	Kxg2	Kxg2	49		
20	Bd3	Nc6	50		
21	Bd1	g5	51		
22	a4	Bd7	52		
23	Kd2	e5	53		
24	Kc3	f6	54		
25	Rd1	Rd8	55		
26	Bc4	h5	56		
27	g4	Ke2	57		
28	Od5	g6	58		
29	Oxc6	Oxc6	59		
30	Kxg2	Kxg2	60		

Hình 20: Ảnh ván cờ 2

1	e4	d6	31	Kf1	Bc3
2	d4	Bd7	32	Rd1	Rf7
3	Bc4	d5	33	Bf2	Bd7
4	e3	Bb6	34	Rh4	Be8
5	Ne2	Bb7	35	Bd5	Bd5
6	e5	Nf6	36	g4	Bd7
7	dxc5	Oxc5	37	Be1	Bd5
8	Bc3	d6	38	Be4	Bd5
9	Qb3	Qb6	39	Rd4	Kf7
10	Bxf7	Kf8	40	Rd4	Kf7
11	Qxb6	Qxb6	41	Bd5	Kf7
12	Bb3	Nc6	42	Bc6	Kf7
13	Ng2	Nd5	43	Bd1	Rd5
14	Bc2	b5	44	Rd1	Bd5
15	O-O	Kf7	45	Bd5	Rd5
16	Rd1	Nd6	46	Rd5	Rd5
17	Bd1	Rd8	47	Rd5	Rd5
18	Q4	Qd3	48	Rd5	Rd5
19	B3	Nd5	49	Rd5	Rd5
20	Rd2	Ng8	50	Rd5	Rd5
21	Nf1	Nc7	51	Rd5	Rd5
22	Bf1	Nc7	52	Rd5	Rd5
23	Ne3	Nc7	53	Rd5	Rd5
24	Nd5	Nd5	54	Rd5	Rd5
25	Oxd5	Bc8	55	Rd5	Rd5
26	Bc3	Nd5	56	Rd5	Rd5
27	Nc3	Nd5	57	Rd5	Rd5
28	Qd5	g5	58	Rd5	Rd5
29	Re2	g4	59	Rd5	Rd5
30	Bc2	Rd7	60	Rd5	Rd5

Hình 21: Ảnh ván cờ 3

1	c4	e5	31		
2	Nc3	Nc6	32		
3	g3	Bb7	33		
4	g3	Nf6	34		
5	O-O	O-O	35		
6	g4	g4	36		
7	g4	g4	37		
8	Nf3	Bd6	38		
9	d3	O-O	39		
10	O-O	Qd7	40		
11	Nd5	Ng4	41		
12	Nd4	Ng4	42		
13	Nf5	Qc5	43		
14	Ne3	d5	44		
15	Nxg4	Qc	45		
16	Qc	Qxc7	46		
17	Nxc7	Qxc7	47		
18	Qxc7	Qxc7	48		
19	Qd7	Qd7	49		
20	Qd7	Qd7	50		
21	Qd7	Qd7	51		
22	Qd7	Qd7	52		
23	Qd7	Qd7	53		
24	Qd7	Qd7	54		
25	Qd7	Qd7	55		
26	Qd7	Qd7	56		
27	Qd7	Qd7	57		
28	Qd7	Qd7	58		
29	Qd7	Qd7	59		
30	Qd7	Qd7	60		

Hình 22: Ảnh ván cờ 4

2.2. Quy trình xử lý

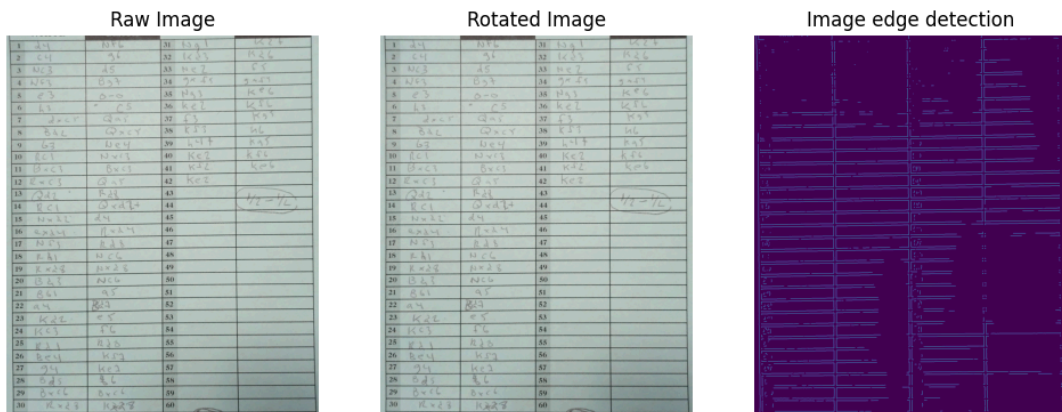
Trong phần này, quy trình xử lý cũng sẽ được chia thành 3 phần với nhiệm vụ như trên.

2.2.1. Tiền xử lý

Phần tiền xử lý sẽ gồm các bước sau

- Áp dụng những phép biến đổi cơ bản: chuyển sang ảnh xám, áp dụng bộ lọc Gaussian để làm tròn, tăng sáng, tăng tương phản.
- Để tăng cường cho bước xử lý chính, ta phải xoay ảnh sao cho các trục vuông góc với biên ảnh. Áp dụng phương pháp biến đổi affine để xoay ảnh theo một góc và để xác định góc quay, thực hiện các bước sau:

- Sử dụng phương pháp Canny để phát hiện cạnh với $T_{lower}=178$, $T_{upper}=200$ và $apertureSize=3$. Việc đặt ngưỡng cao và hẹp như vậy mục đích là để lọc đi những cạnh của chữ, tránh nhiễu.
- Tiếp theo, sử dụng phương pháp Hough để phát hiện những đường thẳng. Những đường thẳng này sẽ được dùng để phát hiện hướng chủ đạo của ảnh. Ở đây, những đường thẳng có hướng thẳng sẽ được loại đi, để lại những đường có hướng ngang. Điều này để lấy ra góc của các đường theo phương ngang hợp với trục x. Ảnh sẽ được quay ngược góc này để căn chỉnh lại ảnh

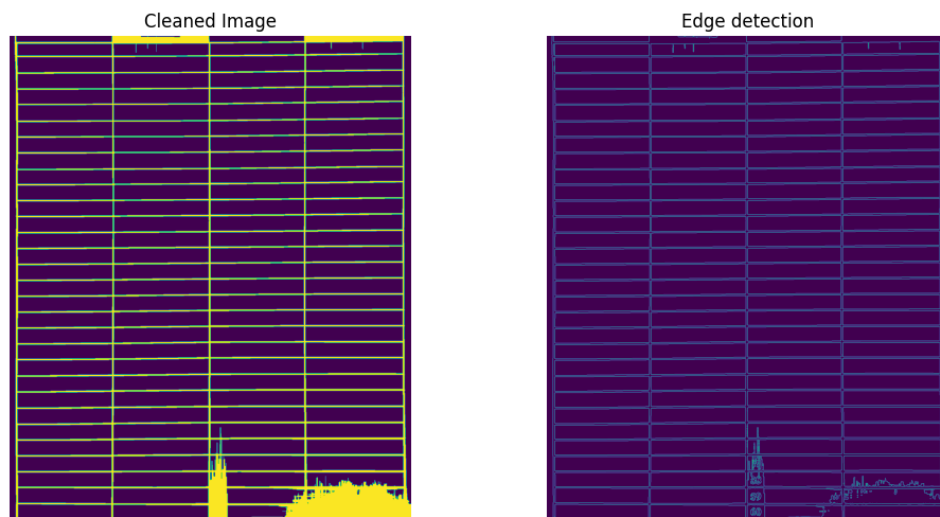


Hình 23: Ảnh trước và sau khi căn chỉnh lại

2.2.2. Xử lý chính

Để cắt được những ô một cách chính xác. Ta cần phát hiện chính xác các đường biên ngang và dọc. Các bước xử lý như sau:

- Chuyển ảnh về dạng nhị phân
- Sử dụng phép mở với kernel giá trị = 1 kích thước (1, n) cho phát hiện biên dọc và (n, 1) cho phát hiện biên ngang. Khi sử dụng phép mở (co rỗng dẫn) với kernel như vậy, các vật thể không phải liên tiếp theo một hướng dọc hoặc ngang sẽ bị lọc đi bởi phép co và những đường thẳng liên tiếp hay những đường kẻ dài trên ảnh sẽ được tăng cường bởi phép dẫn.
- Sau đó áp dụng Canny để phát hiện biên. Hình 24 biểu diễn ảnh sau khi làm rõ đường kẻ và phát hiện biên



Hình 24: Ảnh được tạo ra sau các bước xử lý

2.2.3. Hậu xử lý

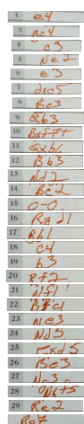
Ảnh sau khi được phát hiện biên và được quay thẳng thì những ô cờ sẽ có hình dạng ô hình chữ nhật. Lúc này ta chỉ cần phát hiện hình chữ nhật và đặt ngưỡng trên, ngưỡng dưới sao cho đúng với 1 ô ghi cờ và cắt ảnh ra. Hình ảnh các ô được cắt:



Hình 25: Các ô nước cờ 1



Hình 26: Các ô nước cờ 2



Hình 27: Các ô nước cờ 3



Hình 28: Các ô nước cờ 4

2.3. Kết quả

Kết quả sau khi cắt ảnh ra như sau:

- Hình 25 thu được 113 ô. Trong đó 62/62 ô có nước cờ phát hiện được.
- Hình 26 thu được 114 ô. Trong đó 83/87 ô có nước cờ phát hiện được.
- Hình 27 thu được 116 ô. Trong đó 107/111 ô có nước cờ phát hiện được.
- Hình 28 thu được 120 ô. Trong đó 57/57 ô có nước cờ phát hiện được.

Phân tích kết quả thu được:

- Một số ảnh vẫn còn bị tối góc hoặc có nhiễu nên khi áp phép mở với bộ lọc như trên bị một phần của ảnh nhị phân chưa được lọc.
- Phép xoay và phát hiện góc chưa thực sự tốt dẫn đến việc ảnh chưa được thẳng ở 1 số ảnh.

3. Content-based Retrieval Image (CBIR)

3.1. Bài toán

Với sự phát triển bùng nổ của dữ liệu đa phương tiện như ảnh, video, âm thanh trên internet, việc tra cứu bằng hình ảnh và camera là việc rất thiết thực trong cuộc sống hàng ngày. Năm 2017, Google đã giới thiệu hệ thống Google lens cho phép người dùng truy vấn trang web chứa ảnh tương tự với ảnh người dùng cung cấp.

Vì vậy, trong khuôn khổ mini project môn Thị giác máy tính, tôi sẽ xây dựng một ứng dụng truy vấn ảnh dựa trên nội dung. Đối với mỗi câu truy vấn, hệ thống cần trả ra K ảnh gần nhất.

3.2. Nghiên cứu liên quan

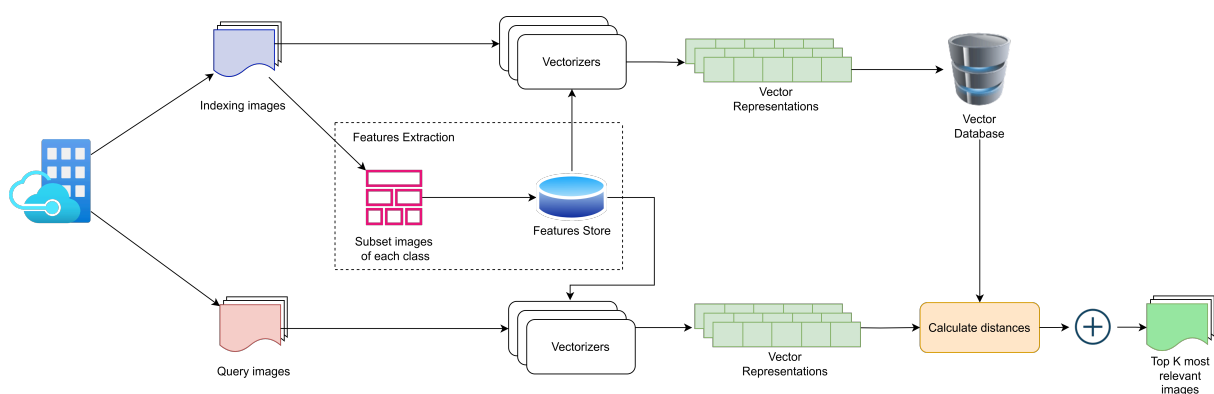
Bài toán tìm kiếm ảnh không phải là bài toán mới và đã có rất nhiều nghiên cứu liên quan đạt được kết quả rất tốt. Những phương pháp [1], [2], [3] xây dựng bộ từ điển hình ảnh và dựa vào đó để tìm kiếm. Gordo [4], Sun [5], Tolias [6] đề xuất ứng dụng deep learning trong việc phân tích các đặc trưng.

3.3. Phương pháp

3.3.1. Ý tưởng

Trong một hệ thống tìm kiếm nói chung, đầu tiên ta cần thực hiện đánh chỉ mục cho những dữ liệu cần tìm kiếm. Chỉ mục ở đây có thể biểu diễn là những từ trong câu đối với tìm kiếm văn bản bằng thuật toán inverted index, hoặc cũng có thể là một vector số đại diện cho văn bản đó.

Với hình ảnh thì ta sẽ lựa chọn đánh chỉ mục bằng cách biểu diễn hình ảnh dưới dạng vector số và tìm kiếm bằng cách tính khoảng cách hoặc độ tương đồng giữa hai vector. Ý tưởng chung được thể hiện trên Hình 29. Đầu tiên, tập dữ liệu được lựa chọn sẽ chia làm 2 phần là phần để tìm kiếm và phần để thử nghiệm. Các ảnh sẽ được vector hóa bằng 1 hoặc nhiều phương pháp và tính toán khoảng cách giữa các vector với nhau để chỉ ra được những ảnh có độ tương đồng tốt nhất bằng thuật toán K-Nearest-Neighbors. Những ảnh tương đồng này sẽ được tổng hợp có trọng số giữa các phương pháp để chỉ ra ảnh tương đồng nhất. Tùy vào phương pháp lựa chọn, một phần nhỏ của tập tìm kiếm sẽ được tách ra để xây dựng kho đặc trưng, kho đặc trưng có thể là từ điển hình ảnh.



Hình 29: Ý tưởng chung cho bài toán tìm kiếm ảnh

3.3.2. Các phương pháp vector hóa

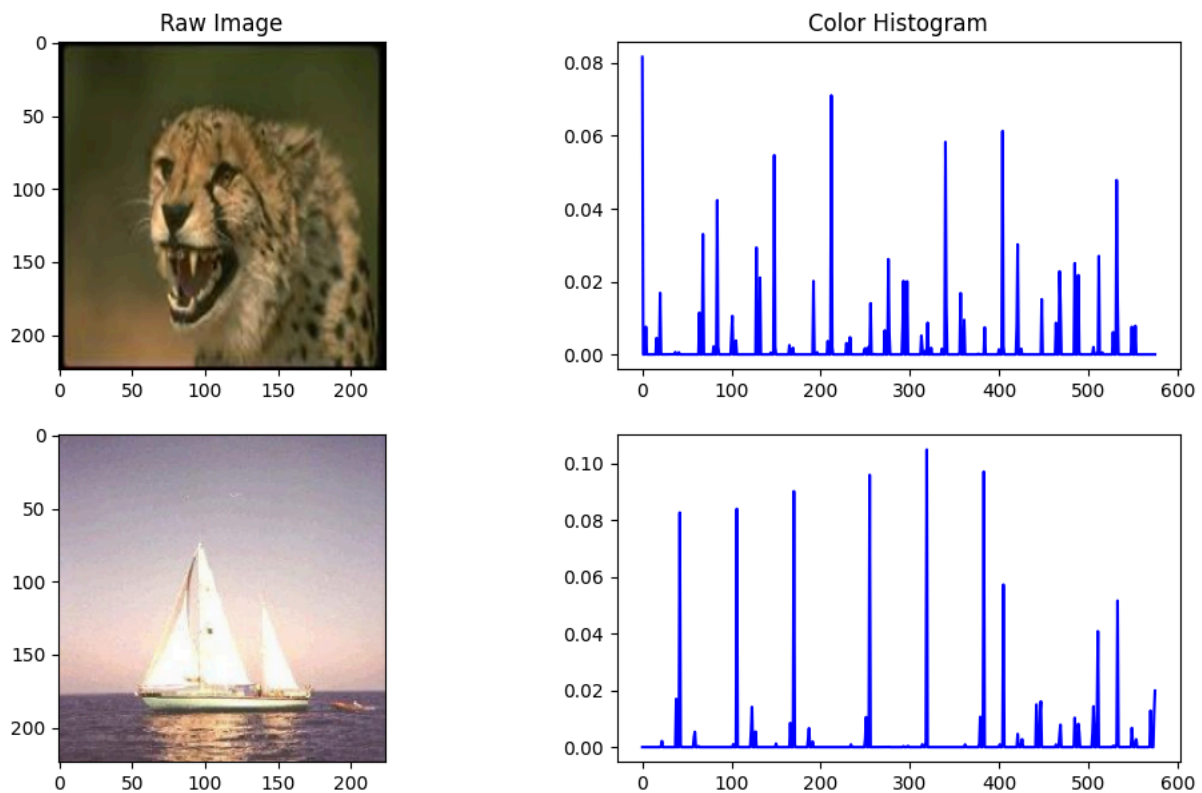
3.3.3. Vector hóa với Histogram màu

Biểu diễn hình ảnh bằng cách đo histogram màu trên 3 kênh: đỏ, xanh lá, xanh biển. Với mỗi điểm ảnh sẽ có các giá trị màu (đỏ, xanh lá, xanh biển), mỗi bộ 3 giá trị này sẽ được biểu diễn là một cột trong histogram màu, giá trị cột là số lần xuất hiện bộ 3 giá trị trong ảnh. Vì số chiều nếu lấy 256 giá trị mỗi kênh màu thì sẽ phải biểu diễn vector với độ dài $256^3 = 16777216$ là cực kỳ lớn. Vì vậy, số lượng cột sẽ được giới hạn tối đa là 12 cột.

Các bước thực hiện:

- Lượng tử hóa màu: Chia không gian màu (ví dụ: RGB, HSV) thành các khoảng màu nhỏ hơn.
- Tạo histogram: Đếm số lượng pixel thuộc vào mỗi khoảng màu, tạo thành một vector biểu diễn histogram màu.
- Chuẩn hóa histogram: Chuẩn hóa histogram để đảm bảo các histogram khác nhau có thể so sánh được.

Ở đây, để tăng hiệu quả biểu diễn, ta sẽ chia khung hình ra thành $n \times m$ phần và tính histogram trên mỗi phần. Sau đó các vector trên từng phần sẽ được nối với nhau tạo thành vector biểu diễn của cả ảnh. Với cách làm này thì ta có thể tối ưu biểu diễn trên các vùng cục bộ thay vì hoàn toàn là toàn cục. Ảnh Hình 30 thể hiện biểu diễn bằng histogram màu với khung hình tỉ lệ 2x2.



Hình 30: Biểu diễn ảnh bằng histogram màu

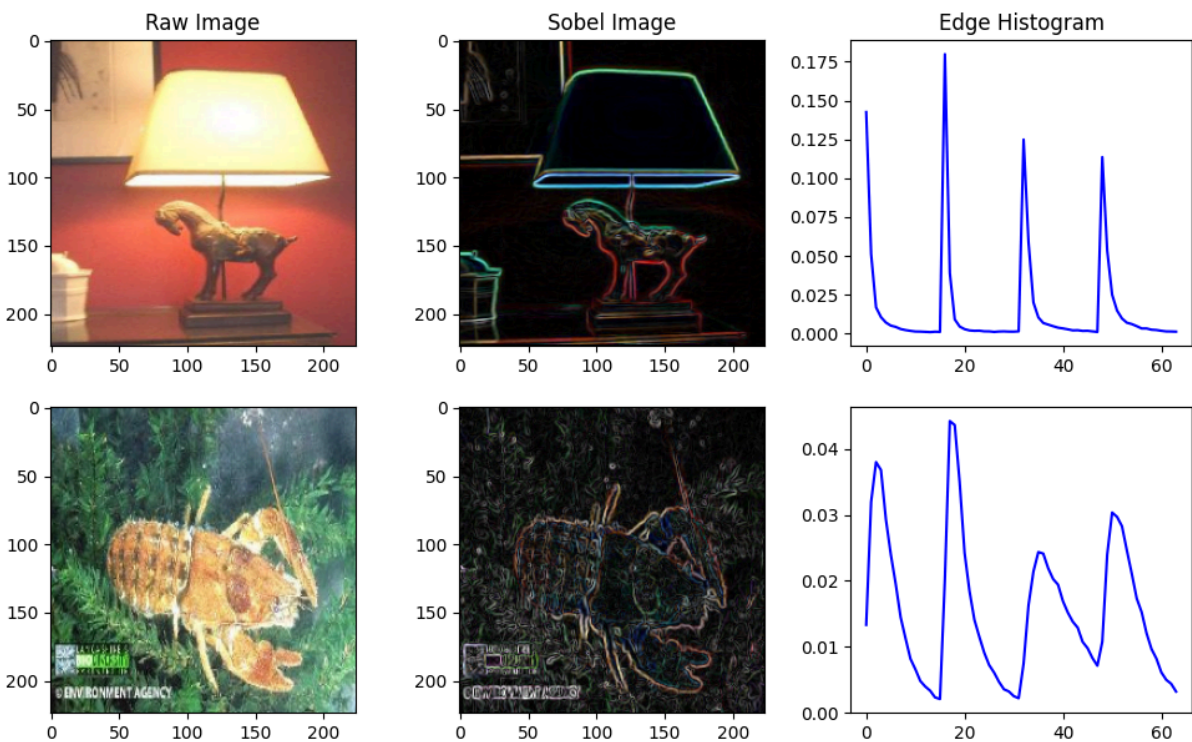
3.3.4. Vector hóa với Histogram cạnh

Thuật toán này dùng để mô tả toàn cục một hình ảnh dựa trên phân bố các cạnh trong hình ảnh đó. Thay vì chỉ quan tâm đến vị trí cụ thể của các cạnh, thuật toán này tập trung vào việc thống kê số lượng các cạnh có độ lớn khác nhau trong toàn bộ hình ảnh. Như vậy những đặc trưng về kết cấu sẽ được thể hiện rất tốt với cách mã hóa này.

Các bước thực hiện:

- Tính toán gradient: Sử dụng các toán tử vi phân (Sobel, Laplacian) để tính toán độ lớn gradient tại mỗi pixel.
- Tạo histogram: Xây dựng một histogram để biểu diễn số lượng pixel có độ lớn gradient nằm trong các khoảng giá trị khác nhau.
- Chuẩn hóa histogram: Chuẩn hóa histogram để đảm bảo các histogram khác nhau có thể so sánh được.

Để tăng hiệu quả biểu diễn, ta cũng sẽ chia khung hình ra thành $n \times m$ phần như trên. Hình 31 thể hiện biểu diễn bằng histogram màu với khung hình tỉ lệ 2x2.



Hình 31: Biểu diễn ảnh bằng histogram cạnh

3.3.5. Vector hóa bằng phương pháp Fourier Descriptor

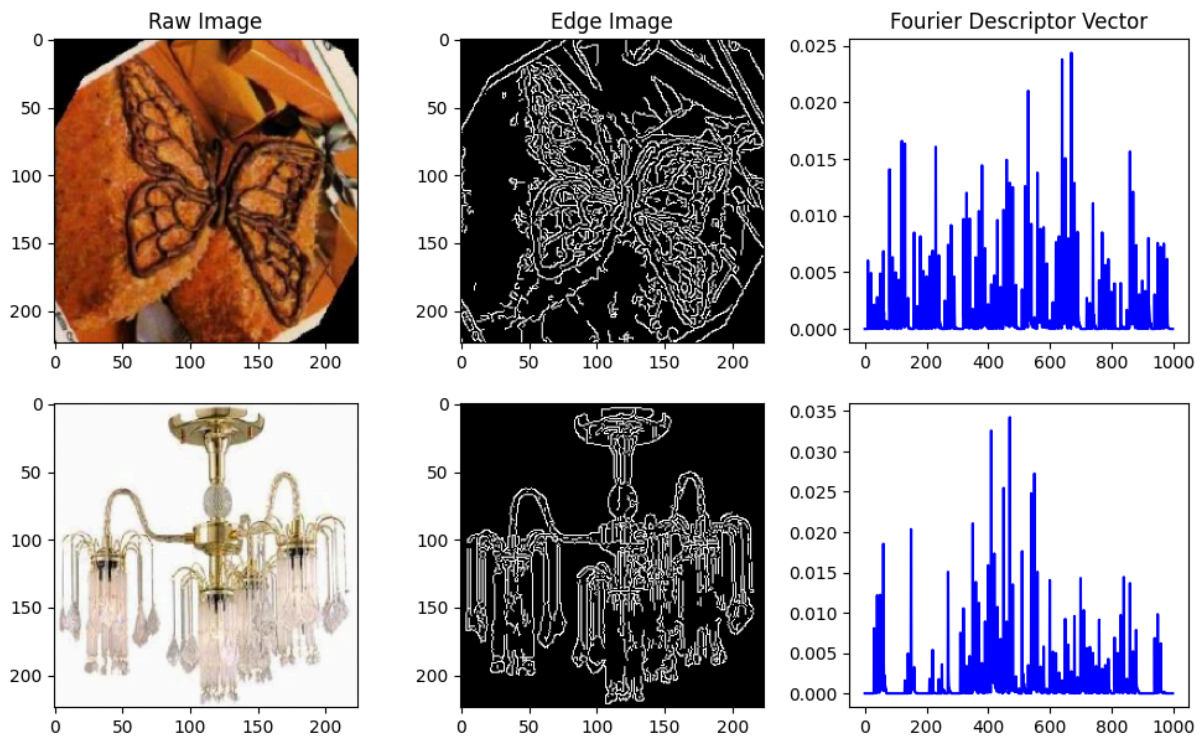
Thuật toán này dùng để mô tả toàn cục một hình ảnh dựa trên phân tích tần số Fourier của đường biên hình ảnh. Thay vì chỉ quan tâm đến vị trí cụ thể của các điểm trên đường biên, thuật toán này tập trung vào việc phân tích các thành phần tần số của đường biên. Với thuật toán này, ta có thể trích xuất được đặc trưng về hình dạng của vật thể và bất biến với tỉ lệ.

Các bước thực hiện:

- Trích xuất đường biên: Sử dụng các thuật toán phát hiện cạnh (ví dụ: Canny) để trích xuất đường biên của hình ảnh.
- Biểu diễn đường biên: Biểu diễn các điểm trên đường biên dưới dạng số phức.

- Biến đổi Fourier: Áp dụng biến đổi Fourier rời rạc (DFT) lên dãy số phức này để thu được các hệ số Fourier.
- Trích xuất đặc trưng: Trích xuất các hệ số Fourier có độ lớn lớn nhất để làm đặc trưng cho hình ảnh.

Để tăng hiệu quả biểu diễn, ta cũng sẽ chia khung hình ra thành $n*m$ phần như trên. Hình 32 thể hiện biểu diễn bằng mô tả fourier với khung hình tỉ lệ 2x2.



Hình 32: Biểu diễn ảnh bằng bộ mô tả fourier

3.3.6. Vector hóa dựa trên bộ mô tả SIFT

Bộ mô tả SIFT

Bộ mô tả SIFT (Scale-Invariant Feature Transform Descriptor) là một trong những thuật toán mô tả được sử dụng phổ biến. Bộ mô tả SIFT được miêu tả là một vector 128 chiều biểu diễn histogram hướng đạo hàm của những điểm xung quanh điểm đặc trưng. Các điểm đặc trưng này được phát hiện bởi thuật toán tìm điểm đặc trưng SIFT. Điểm mạnh của thuật toán mô tả SIFT này là bộ mô tả thu được sẽ là bất biến với tỉ lệ, phép xoay và cường độ chiếu sáng.

Ở đây bộ mô tả SIFT được sử dụng để phát hiện ra những điểm đặc trưng trên ảnh và biểu diễn xung quanh của chúng, mục đích để xây dựng bộ từ điển

Thuật toán Kmeans

Thuật toán Kmeans là một thuật toán phân cụm được sử dụng phổ biến trong rất nhiều bài toán ứng dụng Machine Learning. Thuật toán sẽ hoạt động tốt nhất khi các cụm có hình dạng tròn.

Ở đây, thuật toán Kmeans được sử dụng để gom cụm các đặc trưng tương tự nhau và sử dụng đặc trưng trung tâm của cụm là một “từ” để xây dựng bộ từ điển hình ảnh. Mỗi đặc trưng sẽ được biểu diễn thành “từ” của đặc trưng trung tâm gần nó nhất.

Mô hình túi từ Bag of Words

Để biểu diễn hình ảnh dưới dạng vector, ta sẽ đếm số “từ” trong từ điển xuất hiện chứa trong ảnh đó. Vector sẽ gồm có số chiều bằng số “từ” trong từ điển và giá trị sẽ là số “từ” xuất hiện trong ảnh đầu vào.

Mô hình TF-IDF

Để chuẩn hóa lại vector tạo ra từ mô hình túi từ, ta sẽ sử dụng mô hình TF-IDF. Chỉ số TF-IDF được tính như sau:

$$\text{tfidf}(i, d, D) = \text{tf}(t, d) * \text{idf}(t, D) \quad (1)$$

Trong đó, tf - term frequency biểu thị tần suất xuất hiện của một từ trong văn bản, biểu diễn dưới công thức:

$$\text{tf}(t, d) = \frac{f(t, d)}{(\max\{f(w, d) : w \in d\})} \quad (2)$$

- Thương của số lần xuất hiện 1 từ trong văn bản và số lần xuất hiện nhiều nhất của một từ bất kỳ trong văn bản đó. (giá trị sẽ thuộc khoảng $[0, 1]$)
- $f(t, d)$ – số lần xuất hiện từ t trong văn bản d .
- $\max\{f(w, d) : w \in d\}$ – số lần xuất hiện nhiều nhất của một từ bất kỳ trong văn bản.

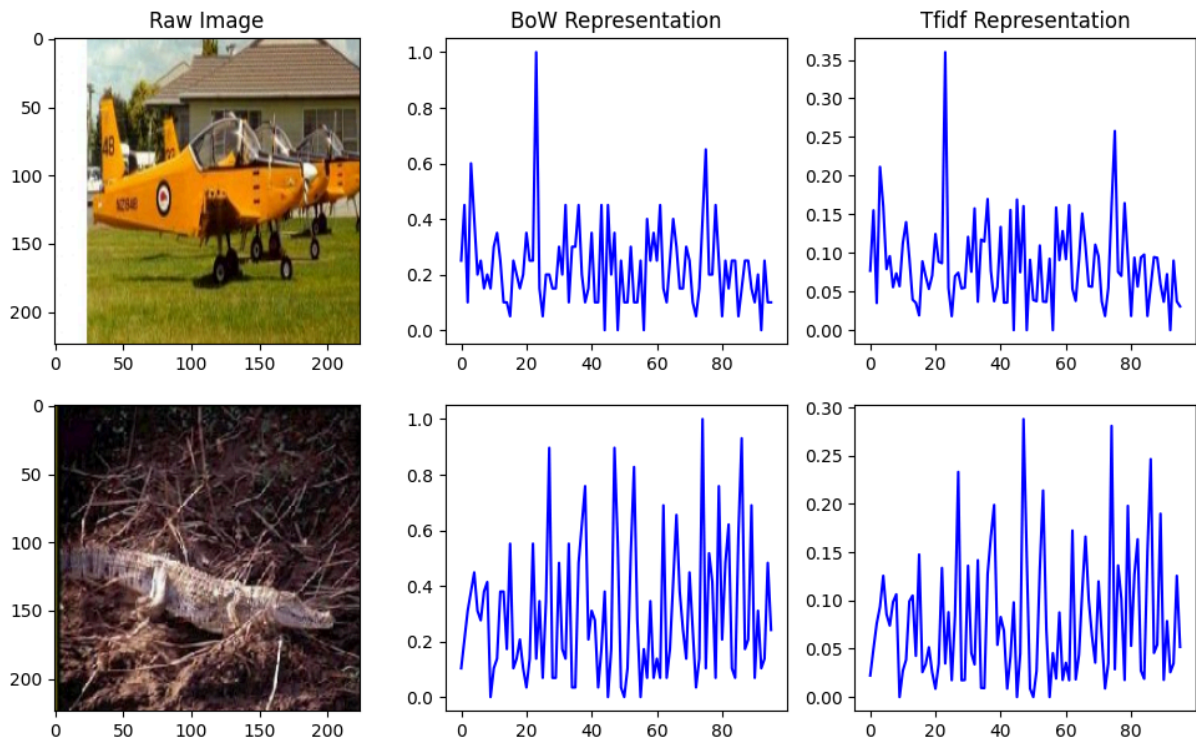
Idf - Inverse document frequency biểu thị tần suất nghịch của một từ trong tập văn bản. Idf được dùng để giảm giá trị những từ phổ biến, biểu diễn với công thức như sau:

$$\text{idf}(t, D) = \log \frac{|D|}{|d \in D : t \in d|} \quad (3)$$

- $|D|$: – tổng số văn bản trong tập D
- $|\{d \in D : t \in d\}|$: – số văn bản chứa từ nhất định, với điều kiện t xuất hiện trong văn bản d . Nếu từ đó không xuất hiện ở bất cứ 1 văn bản nào trong tập thì mẫu số sẽ bằng 0 => phép chia cho không không hợp lệ, vì thế người ta thường thay bằng mẫu thức $1 + |d \in D : t \in d|$.

Ta sẽ coi các “từ” trong từ điển hình ảnh là từ, hình ảnh là văn bản và thực hiện tính toán vector tf-idf cho mỗi biểu diễn ảnh. Tập dữ liệu dùng để tìm kiếm sẽ được chia ra thành 2 tập nhỏ là tập dùng để xây dựng bộ từ điển và tập để tìm kiếm theo tỉ lệ 18 - 82.

Ảnh Hình 33 thể hiện biểu diễn bằng mô tả SIFT và BOW



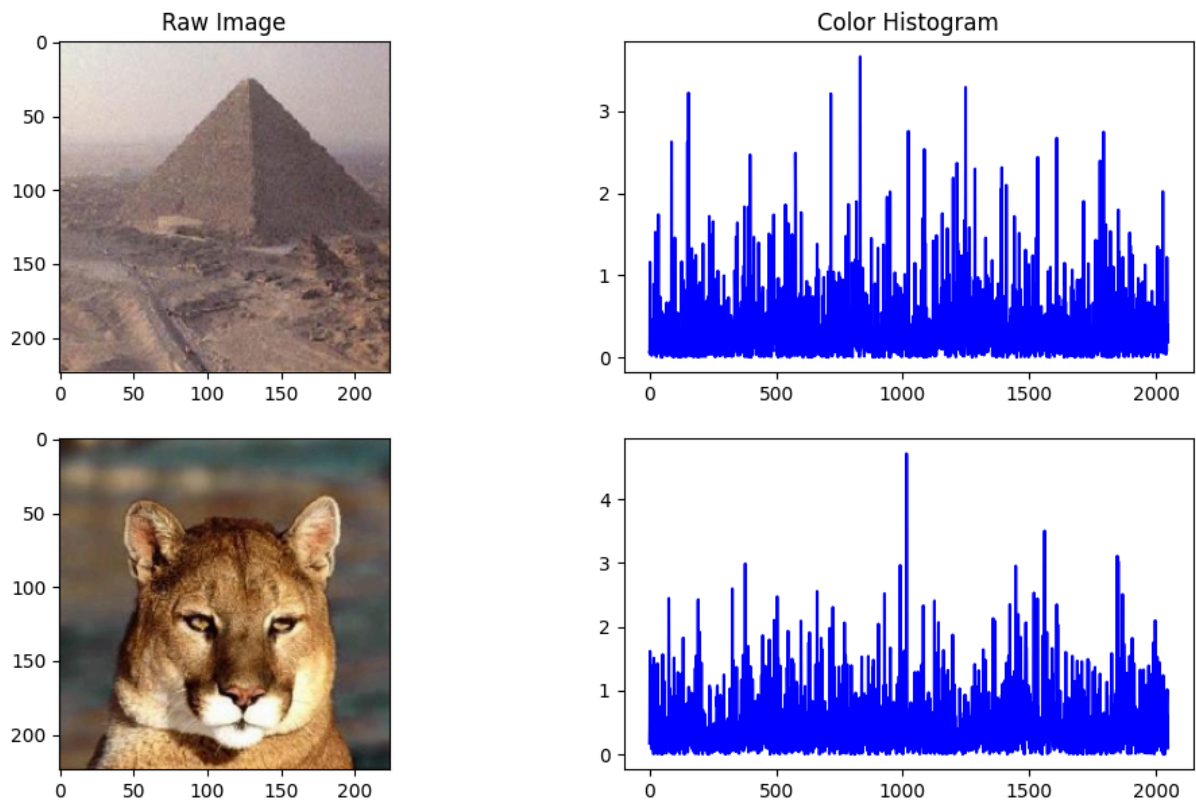
Hình 33: Biểu diễn ảnh bằng bộ mô tả SIFT

3.3.7. Vector hóa hình ảnh bằng Resnet

Với sự phát triển mạnh mẽ của các mạng học sâu trong các bài toán xử lý ảnh, những biểu diễn ẩn giữa các tầng trong mạng học sâu thường được lấy ra để biểu diễn vector cho ảnh.

ResNet (Residual Network) [7] là một kiến trúc mạng nơ-ron sâu, được phát triển để giải quyết vấn đề vanishing gradient khi độ sâu của mạng tăng lên. Điểm đặc trưng của ResNet là các khối residual (còn gọi là skip connections), cho phép dữ liệu có thể bỏ qua một vài tầng và giúp mạng học hiệu quả hơn. ResNet đã chứng tỏ khả năng mạnh mẽ trong bài toán phân loại ảnh trên tập ImageNet và ứng dụng rất nhiều trong việc biểu diễn vector cho ảnh.

Trong đề tài này, tôi đã sử dụng mạng Resnet được huấn luyện sẵn trên tập ImageNet và lấy biểu diễn ẩn trước tầng Fully Connected cuối cùng để biểu diễn vector cho ảnh. Hình 34 thể hiện biểu diễn của ảnh bằng mạng Resnet

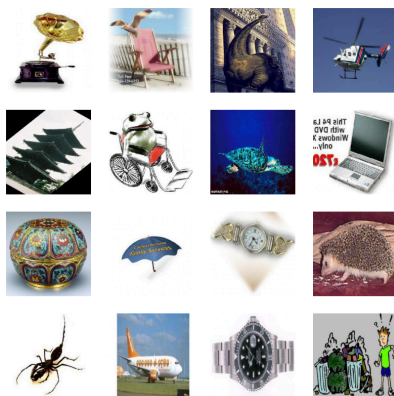


Hình 34: Kiến trúc mô hình Resnet

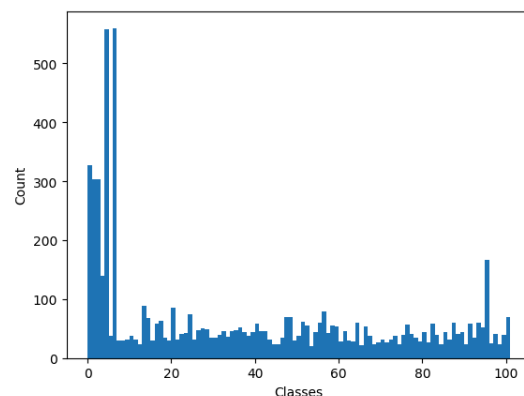
3.4. Thực nghiệm

3.4.1. Bộ dữ liệu

Trong Mini-Project này, tôi lựa chọn bộ Caltech-101 để đánh giá. Caltech-101 thể hiện trong Hình 35 bao gồm khoảng 9.000 hình ảnh được phân loại thành 101 danh mục khác nhau. Các danh mục này rất đa dạng, từ động vật, phương tiện giao thông đến đồ vật hàng ngày. Phân phối dữ liệu được thể hiện trong Hình 36, nhìn chung tương đối là đều nhưng có 1 vài lớp đầu tiên bị mất cân bằng.



Hình 35: Bộ dữ liệu Caltech-101



Hình 36: Phân phối dữ liệu

3.4.2. Phương pháp thực nghiệm

Với mỗi ảnh trên tập thử nghiệm, bộ tìm kiếm sẽ trả ra k ảnh liên quan. Ảnh liên quan được định nghĩa là ảnh có nhãn trùng với nhãn của ảnh đầu vào. Chất lượng của bộ tìm kiếm sẽ

được đánh giá trên ba chỉ số là HitRate@K, MAP@K và Recall@K được biểu diễn bằng Phương trình 4, Phương trình 5, Phương trình 6.

$$\text{HitRate@K} = \left(\frac{1}{U} \right) \sum_{u=1}^U \frac{\sum_{k=1}^K \text{rel}(k)}{K} \quad (4)$$

$$\text{Map@K} = \left(\frac{1}{U} \right) \sum_{u=1}^U \sum_{k=1}^K (P(k) * \text{rel}(k)) \quad (5)$$

$$\text{Recall@K} = \left(\frac{1}{U} \right) \sum_{u=1}^U \frac{\sum_{k=1}^K \text{rel}(k)}{|\text{rel}|} \quad (6)$$

- U là tập ảnh thực nghiệm
- $P(k)$ là chỉ số Precision đo khi ngắt chuỗi tại K
- $\text{rel}(k)$ là hàm chỉ báo bằng 1 nếu mục ở hạng k là nhân có liên quan (chính xác), nếu không thì bằng 0
- $|\text{rel}|$ là số lượng toàn bộ nhân liên quan

Chỉ số HitRate@K dùng để đo số lượng ảnh liên quan trả ra được với top-k đầu ra. Chỉ số Map@K dùng để đánh giá thứ tự trả ra các ảnh liên quan. Chỉ số Recall@K dùng để đánh giá độ phủ của đầu ra so với tập liên quan. Ngoài ra, thời gian để indexing một ảnh và retrieval một ảnh cũng được đánh giá.

3.4.3. Tham số thực nghiệm

Với mỗi phương pháp trích chọn đặc trưng sẽ gồm các tham số thực nghiệm sau:

Color Histogram

- $n_bins = [2, 4, 8, 12]$: số lượng bin với mỗi kênh màu sử dụng để tạo histogram
- $h_types = [\text{"region"}, \text{"global"}]$: chế độ tạo histogram
- $n_slices = [2, 3, 4, 5]$: chia khung hình ra thành $n * n$ phần
- $metrics = [\text{"euclidean"}, \text{"cosine"}]$: hàm đo khoảng cách giữa các vector

Edge Histogram

- $bins = [8, 32, 128]$: số lượng bin với mỗi kênh màu sử dụng để tạo histogram
- $n_slices = [1, 10, 15]$: chia khung hình ra thành $n * n$ phần

Fourier Descriptor

- $num_coeffs = [10, 20, 30]$: số lượng hệ số fourier cần lấy
- $n_slices = [1, 10, 15]$: chia khung hình ra thành $n * n$ phần

SIFT Descriptor and Bag of Words

- $k = [32, 64, 96, 128, 256]$: số lượng từ trong bộ từ điển
- $vector_types = [\text{"tfidf"}, \text{"bow"}]$: phương pháp vector hóa
- $metrics = [\text{"cosine"}, \text{"euclidean"}, \text{"manhattan"}]$: hàm đo khoảng cách giữa các vector

Resnet

- $models = [\text{"resnet18"}, \text{"resnet34"}, \text{"resnet50"}, \text{"resnet101"}, \text{"resnet152"}]$: số lượng lớp trong model resnet được lựa chọn
- $metrics = [\text{"euclidean"}, \text{"cosine"}]$: hàm đo khoảng cách giữa các vector

Ensemble

- d2ss = ["exp", "log", "logistic", "gaussian", "inverse"]: hàm chuyển đổi từ khoảng cách về dạng điểm số từ 0 - 1

- Hàm exp:

$$y = \frac{1}{e^{-x}} \quad (7)$$

- Hàm log:

$$y = -\log(x) \quad (8)$$

- Hàm logistic:

$$y = \frac{1}{1 + e^{-x}} \quad (9)$$

- Hàm gaussian:

$$y = \frac{e^{-x^2}}{2\sigma^2} \quad (10)$$

- Hàm inverse:

$$y = \frac{1}{1 + x} \quad (11)$$

- weights: bộ trọng số ưu tiên với từng phương pháp

3.4.4. Kết quả thực nghiệm

Color Histogram

Thông qua kết quả thực nghiệm được biểu diễn trên Bảng 1. Từ kết quả cho thấy, bộ tham số tối ưu cho phương pháp này là bins=4, type=region, slice=5, metric=cosine.

- Với sự tăng lên của số lượng bins thì thuật toán cho thấy thời gian hoạt động tăng lên đáng kể vì số chiều vector sẽ tăng theo hàm mũ.
- Phương pháp lấy histogram toàn cục cho kết quả không tốt bằng phương pháp cục bộ vì khó biểu diễn được đặc điểm ảnh trên từng vùng khác nhau.
- Khi tăng số vùng thì kết quả được cải thiện hơn tuy nhiên thời gian sẽ tăng nhiều theo hàm lũy thừa với 2.

Bảng 1: Kết quả đánh giá phương pháp color histogram

Bins	Type	Slice	Metric	Map@K			Hit@K			Recall@K			Avg. Time	
				1	5	10	1	5	10	10	100	1000	Index	Retrieve
2	region	2	euclidean	0.11	0.17	<u>0.18</u>	0.21	0.17	<u>0.15</u>	0.01	0.06	<u>0.27</u>	<u>0.000879</u>	0.000923
	global			0.07	0.12	0.13	0.14	0.11	0.1	0.01	0.04	0.25	0.001433	<u>0.000895</u>
		3	euclidean	0.17	0.26	0.28	0.33	0.27	0.24	0.02	0.08	0.31	0.001258	0.004554
	region	4	euclidean	0.17	0.28	0.3	0.35	0.29	0.26	0.02	0.08	0.32	<u>0.00093</u>	0.003416
		5	euclidean	<u>0.18</u>	<u>0.29</u>	<u>0.31</u>	<u>0.36</u>	<u>0.3</u>	<u>0.27</u>	<u>0.02</u>	<u>0.09</u>	<u>0.33</u>	0.001429	<u>0.00333</u>
	4			<u>0.19</u>	<u>0.3</u>	<u>0.31</u>	<u>0.38</u>	<u>0.3</u>	<u>0.27</u>	<u>0.02</u>	<u>0.09</u>	<u>0.33</u>	<u>0.001466</u>	0.008605
8	region	5	euclidean	0.17	0.27	0.28	0.35	0.27	0.23	0.02	0.08	0.31	0.001614	<u>0.006288</u>
12				0.16	0.24	0.25	0.32	0.24	0.2	0.02	0.07	0.3	0.004012	0.026965
4	region	5	cosine	0.2	0.31	0.33	0.4	0.32	0.28	0.02	0.09	0.34	0.001409	0.012869

Edge Histogram

Thông qua kết quả thực nghiệm được biểu diễn trên Bảng 2. Từ kết quả cho thấy, bộ tham số tối ưu cho phương pháp này là bins=8, type=region, slice=10, metric=cosine.

- Với sự tăng lên của số khung hình, thuật toán có cải thiện đáng kể về độ chính xác nhưng sẽ có giới hạn, tăng thêm nữa thì độ chính xác giảm

Bảng 2: Kết quả đánh giá phương pháp edge histogram

Bins	Type	Slice	Metric	Map@K			Hit@K			Recall@K			Avg. Time	
				1	5	10	1	5	10	10	100	1000	Index	Retrieve
8	global		cosine	0.06	0.11	0.12	0.12	0.11	0.1	0.01	0.04	0.27	0.0027	0.002
8	region	15	cosine	0.21	0.34	0.36	0.43	0.36	0.33	0.03	0.11	0.39	0.0212	0.0269
8	region	10	euclidean	0.21	0.34	0.36	0.42	0.36	0.32	0.02	0.11	0.39	0.0109	0.0039
8	region	10	cosine	0.21	0.34	0.36	0.43	0.36	0.33	0.03	0.11	0.39	<u>0.0111</u>	0.0148
32	region	10	cosine	0.21	0.34	0.36	0.43	0.36	0.33	0.03	0.11	0.39	0.0111	0.0146
128	region	10	cosine	0.21	0.34	0.36	0.43	0.36	0.33	0.03	0.11	0.39	0.0111	0.0148

Fourier Descriptor

Thông qua kết quả thực nghiệm được biểu diễn trên Bảng 3. Từ kết quả cho thấy, bộ tham số tối ưu cho phương pháp này là bins=10, type=region, slice=5, metric=cosine.

- Số lượng hệ số Fourier cần lấy hoạt động tốt nhất ở mức 10, tăng lên tuy không có nhiều sự sụt giảm về độ chính xác nhưng thời gian chạy có phần lâu hơn
- Với phương pháp này, càng nhiều khung hình thì thuật toán hoạt động càng tốt

Bảng 3: Kết quả đánh giá phương pháp fourier descriptor

Num coeffs	Type	Slice	Metric	Map@K			Hit@K			Recall@K			Avg. Time	
				1	5	10	1	5	10	10	100	1000	Index	Retrieve
10	global	1	cosine	0.03	0.05	0.05	0.05	0.04	0.04	0.0	0.02	0.17	0.0013	0.0014
10	region	10	euclidean	0.11	0.18	0.19	0.21	0.18	0.16	0.01	0.05	0.29	0.0042	0.002
10	region	10	cosine	0.13	0.22	0.23	0.27	0.22	<u>0.2</u>	0.01	0.07	0.34	0.0042	0.0095
10	region	15	cosine	0.15	0.24	0.25	0.29	0.24	0.21	0.01	0.07	0.35	<u>0.0073</u>	<u>0.0184</u>
20	region	15	cosine	0.14	0.23	0.25	0.29	0.23	0.21	0.01	0.07	0.35	0.0081	0.0296
30	region	15	cosine	0.14	0.23	0.24	0.29	0.23	0.2	0.01	0.07	0.35	0.0087	0.0394

Ensemble Color Histogram, Edge Histogram, Fourier Descriptor

Thông qua kết quả thực nghiệm được biểu diễn trên Bảng 4. Từ kết quả cho thấy, bộ tham số tối ưu cho phương pháp này là weight = (0.2,0.8,0.2).

- Thuật toán hoạt động tốt nhất với bộ trọng số thiên về phương pháp edge histogram.
- Độ chính xác được cải thiện so với 3 thuật toán nhưng không đáng kể và thời gian chạy lâu hơn.

Bảng 4: Kết quả đánh giá phương pháp kết hợp Color và Edge và Fourier

Score Function	Weight (Color, Edge, Fourier)	Map@K			Hit@K			Recall@K			Avg. Time	
		1	5	10	1	5	10	10	100	1000	Index	Retrieve
exp	(0.5, 0.5, 0.5)	0.22	0.34	0.36	0.43	0.36	0.32	0.03	0.11	0.4	0.0162	0.0643
exp	(0.8, 0.2, 0.2)	0.22	0.34	0.36	0.43	0.35	0.32	0.02	0.11	0.37	0.0162	0.0484
exp	(0.2, 0.8, 0.2)	0.23	0.36	0.38	0.45	0.38	0.34	0.03	0.13	0.43	0.0162	0.0484
exp	(0.2, 0.2, 0.8)	0.18	0.29	0.31	0.36	0.3	0.27	0.02	0.09	0.39	0.0162	0.0484
exp	(0.7, 0.7, 0.3)	0.22	0.35	0.37	0.45	0.36	0.33	0.03	0.11	0.39	0.0162	0.0042
exp	(0.3, 0.7, 0.7)	0.2	0.32	0.34	0.4	0.34	0.31	0.02	0.11	0.41	0.0162	0.0484
exp	(0.7, 0.3, 0.7)	0.21	0.34	0.35	0.42	0.35	0.32	0.03	0.11	0.4	0.0162	0.0484

SIFT Descriptor and Bag of Words

Thông qua kết quả thực nghiệm được biểu diễn trên Bảng 5. Từ kết quả cho thấy, bộ tham số tối ưu cho phương pháp này là k=96, type=tfidf, metric=mahanttan.

- Do bộ dữ liệu là hữu hạn, việc tăng lên của kích thước từ điển không những không cải thiện hiệu năng mà còn làm giảm độ chính xác. Tuy nhiên với bộ dữ liệu lớn hơn thì kích thước này cũng cần thay đổi.
- Với cách mã hóa dùng tfidf sẽ tốt hơn bow thông thường vì tfidf có khả năng giảm sức ảnh hưởng của các “từ” phổ biến.
- Khoảng cách manhattan cho hiệu năng tốt nhất với phương pháp này vì vector được biểu diễn tương đối rời rạc, chỉ biểu diễn những “từ” xuất hiện trong ảnh.

Bảng 5: Kết quả đánh giá phương pháp SIFT Descriptor và Bag of Words

K	Type	Metric	Map@K			Hit@K			Recall@K			Avg. Time		
			1	5	10	1	5	10	10	100	1000	Fitting	Index	Retrieve
32	tfidf	cosine	0.1	0.17	0.18	0.21	0.16	0.15	0.01	0.07	0.32	21.66	0.0082	0.0066
		euclidean	0.1	0.17	0.18	0.21	0.16	0.15	0.01	0.07	0.32	21.66	0.0119	0.0103
		manhattan	0.1	0.17	0.18	0.2	<u>0.17</u>	0.15	0.01	0.07	<u>0.34</u>	21.66	<u>0.0083</u>	0.0029
	bow	cosine	0.1	0.17	0.18	0.21	0.16	0.15	0.01	0.07	0.32	21.66	0.0082	0.0057
		euclidean	0.1	0.17	0.18	0.21	0.16	0.15	0.01	0.07	0.32	21.66	0.0116	0.0089
		manhattan	0.1	0.17	0.18	0.2	0.17	0.15	0.01	0.07	0.34	21.66	0.0092	0.0029
64	tfidf	manhattan	0.11	0.18	0.19	0.22	0.17	0.16	0.01	0.08	0.35	31.63	0.0085	0.003
96			0.11	0.18	0.19	0.22	0.18	0.16	0.02	0.07	0.33	40.59	0.0082	0.0029
128			0.11	0.18	0.19	0.22	0.17	0.16	0.01	0.08	0.34	53.84	0.0082	0.0029
256			0.09	0.16	0.17	0.19	0.16	0.14	0.01	0.06	0.31	87.1	0.0085	0.003

Resnet

Thông qua kết quả thực nghiệm được biểu diễn trên Bảng 6. Từ kết quả cho thấy, bộ tham số tối ưu cho phương pháp này là model=resnet101, metric=cosine.

- Mô hình càng to thì hoạt động càng tốt, tuy nhiên với bộ dữ liệu này chỉ cần đạt 101 lớp là có thể biểu diễn ở mức tối ưu. Việc tăng kích thước mô hình quá lớn dẫn đến thời gian chạy lâu hơn mà không cải thiện được hiệu năng.
- Phương pháp sử dụng Resnet cho thấy độ chính xác vượt trội so với 2 phương pháp trên tuy nhiên thời gian chạy sẽ cao hơn khá nhiều nếu chạy trên CPU. Qua thực nghiệm, khi sử dụng CPU, thời gian chạy của mô hình sẽ gấp 10 - 20 lần thời gian chạy với GPU

Bảng 6: Kết quả đánh giá phương pháp Resnet

Model	Metric	Map@K			Hit@K			Recall@K			Avg. Time(GPU)	
		1	5	10	1	5	10	10	100	1000	Index	Retrieve
resnet18	euclidean	0.4	0.66	0.7	0.8	0.75	0.7	0.1	0.42	0.82	0.0011	0.0021
	cosine	<u>0.42</u>	<u>0.68</u>	<u>0.72</u>	<u>0.83</u>	<u>0.77</u>	<u>0.74</u>	<u>0.11</u>	<u>0.47</u>	<u>0.88</u>	0.0011	0.0035
resnet34	cosine	0.43	0.7	0.75	0.85	0.81	0.78	0.12	0.53	0.92	0.0012	0.0036
resnet50		0.43	0.71	0.75	0.86	0.81	0.78	0.12	0.53	0.91	0.0029	0.0141
resnet101		0.44	0.72	0.78	0.88	0.84	0.8	0.13	0.56	0.93	<i>0.0035</i>	<i>0.01</i>
resnet152		0.44	0.72	0.77	0.88	0.83	0.8	0.12	0.56	0.93	0.0043	0.0166

Ensemble SIFT and Color

Thông qua kết quả thực nghiệm được biểu diễn trên Bảng 7. Từ kết quả cho thấy, bộ tham số tối ưu cho phương pháp này là scorefunc=exp và weight = (0.8, 0.2)

- Khi kết hợp 2 phương pháp Sift và Color, ta thấy MAP@10 đã tăng từ 0.33 lên 0.35, Hit@K đã tăng từ 0.28 lên 0.31, Recall@1000 đã tăng từ 0.35 lên 0.36.
- Hàm exp hoạt động tốt nhất khi biểu diễn điểm tuy nhiên là không đáng kể.

Bảng 7: Kết quả đánh giá phương pháp kết hợp SIFT và Color

Score Function	Weight (Sift, Color)	Map@K			Hit@K			Recall@K			Avg. Time	
		1	5	10	1	5	10	10	100	1000	Index	Retrieve
exp	(0.2, 0.8)	0.2	0.31	0.33	0.4	0.32	0.29	0.02	0.09	0.34	0.009	0.0285
	(0.4, 0.6)	0.2	0.32	0.34	0.4	0.33	0.29	0.02	0.1	0.34	0.009	0.0285
	(0.5, 0.5)	0.21	0.32	0.34	0.41	0.33	0.29	0.02	0.1	0.34	0.009	0.0285
	(0.6, 0.4)	0.21	0.33	0.34	0.42	0.33	0.3	0.02	0.1	0.35	0.009	0.0285
	(0.8, 0.2)	0.21	0.33	0.35	0.41	0.35	0.31	0.03	0.11	0.36	0.009	0.0285
log	(0.8, 0.2)	0.21	0.33	0.35	0.41	0.35	0.31	0.03	0.11	0.36	0.009	0.0285
logistic	(0.8, 0.2)	0.21	0.33	0.35	0.41	0.35	0.31	0.03	0.11	0.36	0.009	0.0285
gaussian	(0.8, 0.2)	0.21	0.33	0.35	0.41	0.35	0.31	0.03	0.11	0.36	0.009	0.0285
inverse	(0.8, 0.2)	0.21	0.33	0.35	0.41	0.35	0.31	0.03	0.11	0.36	0.009	0.0285

Ensemble SIFT and Color and Resnet

Thông qua kết quả thực nghiệm được biểu diễn trên Bảng 8. Từ kết quả cho thấy, bộ tham số tối ưu cho phương pháp này là weight=(0.16, 0.04, 0.8). Ta thấy khi kết hợp resnet với 2 phương pháp còn lại, hiệu năng không những không được cải thiện mà còn sụt giảm. Điều này cho thấy hiệu quả vượt trội của các phương pháp deep learning trong xử lý ảnh.

Bảng 8: Kết quả đánh giá phương pháp kết hợp SIFT và Color và Resnet

Score Function	Weight (Sift, Color, Resnet)	Map@K			Hit@K			Recall@K			Avg. Time	
		1	5	10	1	5	10	10	100	1000	Index	Retrieve
exp	(0.16, 0.04, 0.8)	0.44	0.72	0.77	0.87	0.84	0.8	0.13	0.56	0.93	0.0123	0.0484
exp	(0.32, 0.08, 0.6)	0.43	0.72	0.77	0.87	0.83	0.79	0.12	0.54	0.92	0.0123	0.0484
exp	(0.4, 0.1, 0.5)	0.43	0.71	0.76	0.86	0.81	0.78	0.12	0.52	0.91	0.0123	0.0484
exp	(0.48, 0.12, 0.4)	0.42	0.69	0.74	0.85	0.79	0.76	0.11	0.49	0.89	0.0123	0.0484
exp	(0.64, 0.16, 0.2)	0.38	0.62	0.66	0.76	0.69	0.64	0.09	0.35	0.76	0.0123	0.0484

3.5. Kết luận

Trong báo cáo này, tôi đã thử nghiệm 5 phương pháp dùng histogram màu, histogram cạnh, mô tả fourier, sift kết hợp bag of words, resnet để thử nghiệm bài toán tìm kiếm ảnh dựa trên content. Từ kết quả thực nghiệm cho thấy khi kết hợp những phương pháp xử lý ảnh cơ bản là histogram màu, histogram cạnh, mô tả fourier, sift cho thấy có sự cải thiện về hiệu năng. Những phương pháp ứng dụng deep learning vẫn cho thấy sức mạnh vượt trội hơn nhiều tuy nhiên có sự đánh đổi về mặt thời gian và khối lượng tính toán.

Tài liệu tham khảo

- [1] Sivic và Zisserman, “Video Google: A text retrieval approach to object matching in videos”, trong *Proceedings ninth IEEE international conference on computer vision*, 2003, tr 1470–1477.
- [2] J. Philbin, O. Chum, M. Isard, J. Sivic, và A. Zisserman, “Object retrieval with large vocabularies and fast spatial matching”, trong *2007 IEEE conference on computer vision and pattern recognition*, 2007, tr 1–8.
- [3] G. Csurka, C. Dance, L. Fan, J. Willamowski, và C. Bray, “Visual categorization with bags of keypoints”, trong *Workshop on statistical learning in computer vision, ECCV*, 2004, tr 1–2.
- [4] A. Gordo, J. Almazán, J. Revaud, và D. Larlus, “Deep image retrieval: Learning global representations for image search”, trong *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part VI 14*, 2016, tr 241–257.
- [5] J. Sun, Z. Shen, Y. Wang, H. Bao, và X. Zhou, “LoFTR: Detector-free local feature matching with transformers”, trong *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, tr 8922–8931.
- [6] G. Tolias, T. Jenicek, và O. Chum, “Learning and aggregating deep local descriptors for instance-level recognition”, trong *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part I 16*, 2020, tr 460–477.
- [7] K. He, X. Zhang, S. Ren, và J. Sun, “Deep residual learning for image recognition”, trong *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, tr 770–778.

Index of Figures

Hình 1: Ảnh gốc	3
Hình 2: Ảnh có nhiều sọc trắng đen	3
Hình 3: Ảnh có nhiều hạt tiêu	3
Hình 4: Ảnh có cường độ sáng không đều	3
Hình 5: Ảnh trước khi lọc nhiễu	4
Hình 6: Ảnh sau khi lọc nhiễu	4
Hình 7: Ảnh có nhiều sọc đen trắng	4
Hình 8: Miền tần số của ảnh	4
Hình 9: Miền tần số của ảnh khi chuyển gốc tọa độ về trung tâm	4
Hình 10: Zoom vào trung tâm miền tần số	5
Hình 11: Ảnh sau khi lọc nhiễu sọc	5
Hình 12: Ảnh nhị phân	5
Hình 13: Ảnh nhị phân sau khi thực hiện phép mở	5
Hình 14: Ảnh nhị phân sau khi thực hiện phép co	5
Hình 15: Ảnh đã qua xử lý của ảnh thường Hình 1	6
Hình 16: Ảnh đã qua xử lý của ảnh sọc trắng đen Hình 2	6
Hình 17: Ảnh đã qua xử lý của ảnh nhiễu hạt tiêu Hình 3	6
Hình 18: Ảnh đã qua xử lý của ảnh sáng không đều Hình 4	6
Hình 19: Ảnh ván cờ 1	8
Hình 20: Ảnh ván cờ 2	8
Hình 21: Ảnh ván cờ 3	8
Hình 22: Ảnh ván cờ 4	8
Hình 23: Ảnh trước và sau khi căn chỉnh lại	9
Hình 24: Ảnh được tạo ra sau các bước xử lý	9
Hình 25: Các ô nước cờ 1	10
Hình 26: Các ô nước cờ 2	10
Hình 27: Các ô nước cờ 3	10
Hình 28: Các ô nước cờ 4	10
Hình 29: Ý tưởng chung cho bài toán tìm kiếm ảnh	11
Hình 30: Biểu diễn ảnh bằng histogram màu	12
Hình 31: Biểu diễn ảnh bằng histogram cạnh	13
Hình 32: Biểu diễn ảnh bằng bộ mô tả fourier	14
Hình 33: Biểu diễn ảnh bằng bộ mô tả SIFT	16
Hình 34: Kiến trúc mô hình Resnet	17
Hình 35: Bộ dữ liệu Caltech-101	17
Hình 36: Phân phối dữ liệu	17